Dt.: Pg.: Delta

Destructors.

is a special class function which destroys the object as soon as the scope of object ends. The destructor is called automatically by compiler when object goes out of scope.

Syntax of destructor is same as that for constructor with a tidle (~) sign as prefix.

```
class A
{
  public:
    ~A();
};
```

destructor → per class, one destructor can be used
→ for freeing memory
→ automatically called.

Namespace

is a container for identifiers. It puts the name of its members in a distinct space so that they don't conflict with names in other namespaces or global namespace.

```
namespace myspace
{
   // declarations
}
int main()
{

}
```

This will create a namespace called myspace, inside which you can create member declaration.

Rules to create namespace.

(i) namespace definition must be done at global scope or nested inside another namespace.

(ii) namespace definition doesn't terminate with a semicolon like class.

(iii) You cannot create instance of namespace.

(iv) There can be unnamed namespace too. Unnamed namespace is unique for each translation unit.

Using a Namespace.

(i) Scope resolution

(ii) Using directive

(iii) Using declaration

Static keyword.

- are allocated storage only once in a program lifetime in static storage area.

(i) static variables inside functions.

- are initialized only once and they hold there value even through function calls.

with static

```
void counter()
{
    static int count = 0;
    cout << count++;
}
int main()
{
    for(int i = 0; i<5; i++)
    {
        counter();
    }
}
```

output: 0 1 2 3 4

without static

```
void counter()
{
    int count = 0;
    cout << count++;
}
int main()
{
    for(int i = 0; i<5; i++)
    {
        count();
    }
}
```

output: 0 0 0 0 0

Dt.: Pg.: Delta

(11) static class objects

- objects declared static are allocated storage in static storage area and have scope till end of program.
- static objects are also intialized using constructors like other normal objects. Assignment to zero, on using static keyword is only for primitive datatypes, not for user defined datatypes.

```
class Abc
{
  int i;
  public:
  Abc()
  {
    i=0;
    cout << " constructor";
  }
  ~Abc()
  {
    cout << " destructor";
  }
void f()
 {
   static Abc obj;
 }
int main()
 {
   int x = 0;
   if (x == 0)
   {
     f();
   }
  cout << " END";
}
```

output: constructor END
destructor

(III) Static data member in class.

- are those members which are shared by all the objects. Static data member has a single piece of ~~government~~ storage, and is not available as seperate copy with each object, like ~~one~~ other non-static data members.

- static member variables (data members) are not initialized using constructor, because these are not dependent on object initialization.

- Also, it must be initialized explicitly, always outside the class. If not initialized, Linker will give error.

```
class X
{
    static int i;
    public :
      X()
      { };
};
int main()
{
    X obj;
    cout << obj.i;                    // prints value of i
}
```

Once the definition for static data members is made, user cannot redefine it. Though, arithmetic operations can be performed on it.

(IV) Static member function.

These functions work for class as whole rather than for a particular object of a class.

It can be called using an object and direct member access (.) operator. But, its more typical to call a static member function by itself, using class name and scope resolution (::) operator.

Example:

```
class X
{
  public:
    static void f()
    { };
};
int main()
{
  X::f();          // calling member func^n directly with
}                  class name.
```

These func^n cannot access ordinary data members and member functions, but only static data members and static member functions.

→ Constant Keyword.
- which doesnot change.

(1) Constant variables.
- you cannot change value of constant variable.
- constant variable must be intialized when declared.

```
int main()
{
  const int i = 10;
  Const int j = i + 10;        // works fine
  i++;                         // compile time error.
}
```

(II) Pointers with constant

can be done in two ways
- apply const to what pointer is pointing to
- we can make pointer itself a const.

(I) Pointer to Const

```
const int *u;
int const *u;
```
} means same.

u is a pointer that points to a const int

(II) Const pointer

To make pointer const, we have to put const keyword to right of the (*).

```
int x = 1;
int * const w = &x;
```

Here, w is a pointer, which is constant, that points to a int.
Now we cannot change pointer but can change value that it points to.

(III) Constant Function Arguments and Return types.

```
void f( const int i)
{
  i++;          //error
}

const int g ()
{
  return 1;
}
```

Dt.:
Pg.:

(iv) const class Data members.

These are data variables in class which are made const. They are not intialized during declaration. There intilization is occurs in constructor.

```
class Test
{
    const int i;
    public:
        Test (int x)
        {
            i = x;
        }
};
int main()
{
    Test t(10);      ✓      // every object, i is independent
    Test t1(20);     ✓      copy is present.
}
```

once intilized, it cannot be changed.

(v) Constant class object and const member function.

A constant member function never modifies data members in an object.

```
class X
{
    int i;
    public:
        X (int x)                    // constructor
        { i = x;
        }
```

```
    int f() const                    // constant function
    {  i++;
    }

    int g()
    {  i++;
    }
};

int main()
{
    X obj1(10);                      // non const object
    const X obj2(20);                // const object

    obj1.f();                        // no error
    obj2.f();                        // no error
    cout << obj1.i << obj2.i;

    obj1.g();                        // no error
    obj2.g();                        // compile time error
}
```

output : 10 20

→ mutable keyword

is used with member variables of class, which we want to change even if object is of const type.

```
class Z
{
    int i;
    mutable int j;
```

Dt.: Pg.: Delta

```
public:
    z()
    { i=0;
      j=0;
    }
    void f() const
    {
        i++;          // error
        j++;          // works, bcz j is mutable.
    }
};
int main()
{
    const z obj;
    obj.f();
}
```

References in C++

are like constant pointers that are automatically ~~def~~ dereferenced. It is a new name given to an existing storage. So when you are accessing the reference, you are actually accessing that storage.

```
int main()
{
    int y = 10;
    int &r = y;          // r is a reference to y.
    cout<<r;             // output 10
}
```

Dt.: Pg.: Delta

Difference b/w Reference and Pointers.

References	Pointers.
(i) References must be intialized when it is created.	(i) Pointers can be intialized any time.
(ii) once intialized, we cannot reintialize a reference.	(ii) Pointers can be reintialized any time.
(iii) You can never have a NULL reference.	(iii) Pointers can be NULL.
(iv) Reference is automatically dereferenced.	(iv) * is used to dereference a pointer.

const Reference.

is used in function arguments to prevent function from changing the argument.

```
void g ( const int & x)
{  x++;                    // error.
}

int main()
{
   int i = 10;
    g(i);
}
```

we cannot change the argument in function because it is passed as Const reference.

Page No.
Date

→ Pointers

store address of variables or memory location.

syntax

datatype *var_name;

ex: int *ptr;

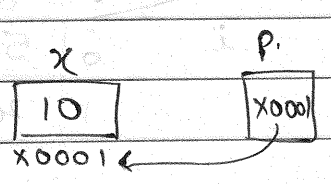
to access address of a variable to a pointer, we use unary operator & (ampersand) that returns address of that variable.

```
int x = 10;                    or    int x = 10;
int *ptr;                            int *ptr = &x;
ptr = &x;
```

[pointer & variable

ex:

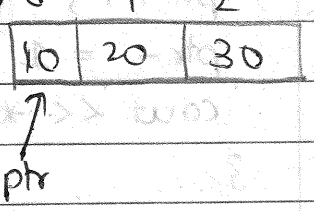
```
int main()
{
    int x = 5;
    int *p = &x;
    cout << x;          // 5
    cout << &x;         // X0001
    cout << *p;         // 5
    cout << p;          // X0001
}
```



pointers and one dimensional array

ex

```
int main()
{ int v[3] = {10, 20, 30};
  int *ptr;
    ptr = v;
for (int i = 0; i < 3; i++)
    { cout << *ptr;          // 10 20 30
      ptr++;
    }
}
```



Page No.
Date

Pointer & two dimensional array

int num[2][3] = { {5, 10, 15}, {20, 25, 30} }

In general, num[i][j] = *(*(num+i)+j)

pointer notation	array notation	value
*(*num)	num[0][0]	5
*(*num+1)	num[0][1]	10
*(*num+2)	num[0][2]	15
((num+1))	num[1][0]	20
((num+1)+1)	num[1][1]	25
((num+1)+2)	num[1][2]	30

i \ j	0	1	2
0	5	10	15
1	20	25	30

eg:

```
void main()
{
  int num[] = {10, 20, 30, 40};
  int *ptr, i;
  ptr = num;
  ptr++;
  ptr -= 1;          // ptr = ptr - 1
  cout << *ptr;
}
```

output : 10

Dt.: Pg.: Delta

Argument Passing Guidelines.

call by value is used during function call just to save our object or variable from being changed or modified, but wherever we pass an argument by value, its new copy is created.

If we pass an object as argument then a copy of that object is created (constructor and destructor called), which affects efficiency. ex: add(a,b) ⟶ int add(int x, int y)

Here, we must use const reference type arguments. When we use, const reference, only an address is passed on stack, which is used inside the function and function cannot change our argument because it is of const type.

So, using const reference type arguments reduces overhead and also saves our argument from being changed.

ex: add(&a, &b) ⟶ void add(int *x, int *y)

call by address example.

```
#include <iostream.h>
void swap (int *, int *);

int main()
{  int a, b;
   cin >> a >> b;
   cout << "before swapping"
        << a << b;
   swap (&a, &b);
   cout << "after swapping"
        << a << b;
   getch(); / return 0;
}
```

```
void swap (int *x, int *y)
{
   int temp;
   temp = *x;
   *x = *y;
   *y = temp;
}
```

[redacted]

Dt.:
Pg.:

Pointers to class members.

- just like pointers to normal variables and functions, we have pointers to class member functions and member variables.

Defining a pointer of class type.

- can used to point to class objects.

```
class simple
{
    public:
        int a;
};
int main()
{
    simple obj;
    simple *ptr;          // pointer of class type.
    ptr = &obj;

    cout << obj.a;
    cout << ptr -> a;     // accessing member with pointers.
}
```

Pointers to data members of class.

we can use pointer to class's data members (member variables)

syntax for declaration: datatype class_name :: *pointer_name;

syntax for assignment: pointer_name = & class_name :: datamember_name;

both declaration and assignment can be done in single statement.

using with objects.

When we have pointer to data member, we have to dereference that pointer to get what its pointing to, hence it becomes.

object. * pointer_to_member

and with pointer to object, it can be accessed by writing

objectpointer → * pointer_to_member.

```
class Data
{
   public:
      int a;
      void print()
      { cout << "a is" << a;
       }
};

int main()
{
   Data d, *dp;
   dp = &d;                          // pointer to object

   int Data :: *ptr = &Data :: a;    // pointer to data member a
   d.*ptr = 10;
   d.print();

   dp → *ptr = 20;
   dp → print();
}
```

output: a is 10
a is 20

Dt.:
Pg.:

Pointers to member functions

pointers can be used to point class's member function.

syntax:

```
return_type (class_name :: *ptr_name)(argument_type) =
                                    &class_name :: function_name;
```

ex:

```
class Data
{
    public :
      int f (float)
        {
          return 1;
        }
};
int (Data :: *fp1) (float) = &Data :: f ;      //declaration and assignment
int (Data :: *fp2) (float);                    //only declaration

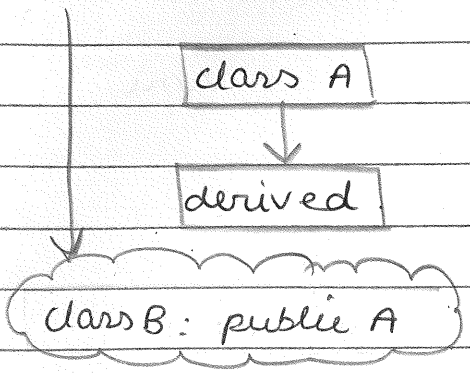
int main ()
{
   fp2 = &Data :: f ;                          //assignment inside main()
}
```

Note:

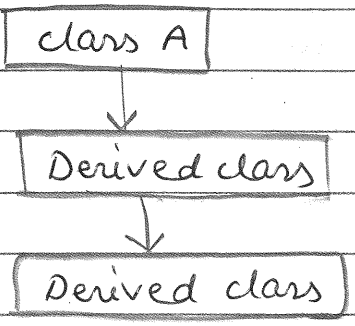
(i) you can change the value and behaviour of these pointer on runtime.

(ii) to have pointer to data member and member func^ you need to make them public ;

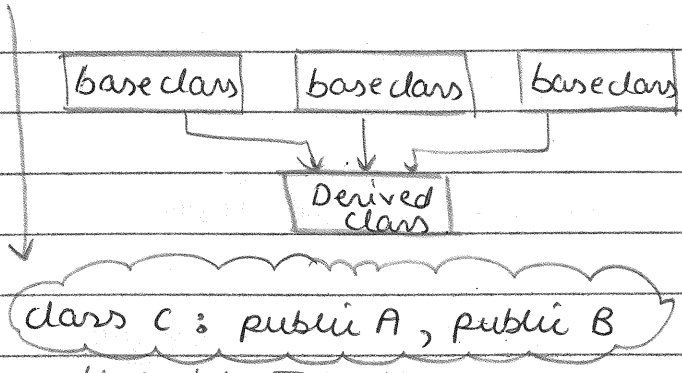
(I) Single Inheritence.



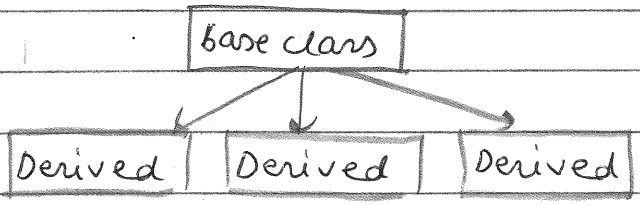
(II) Multilevel Inheritence.



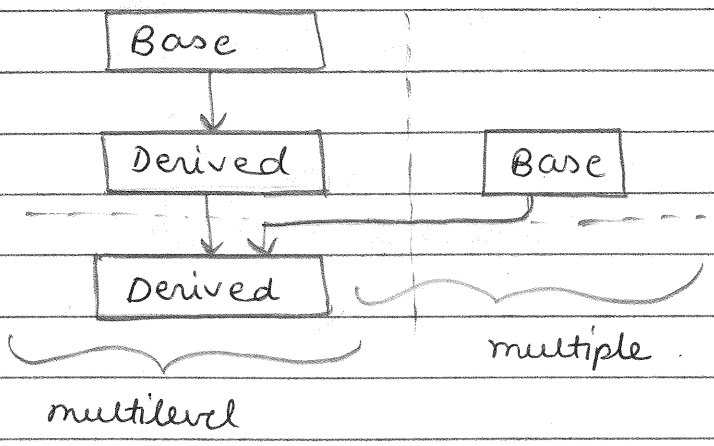
(III) Multiple Inheritence



(IV) Herarichal Inheritence.



(V) Hybrid Inheritence.



Multilevel example.

```
class student
class test : public student
class result : public test
```

Herarichal example.

```
class A
class C : public A
class B : public A
```

Syntax:

```
class B : A
class B : private A
class B : public A
```

} same

Hybrid Inheritence

```
class student
class student : public student
class sport
class result : public test,
               public sport
```

Dt.:
Pg.:
Delta

Inheritance concept.

class derived-classname : visibility-mode base-classname.

↙

public or protected or private.

Test	class A	class B
private	private	private
protected	protected	protected
public	public	public

class A : Public test

(i) cannot inherit private func^n or data

(ii) protected will be inherited in protected part of A.

(iii) public of test will be inherited in public part of A

class B : private test

(i) cannot inherit private data or func^n

(ii) protected of test will be inherited in private of class B.

(iii) public of test will be inherited in private of class B.

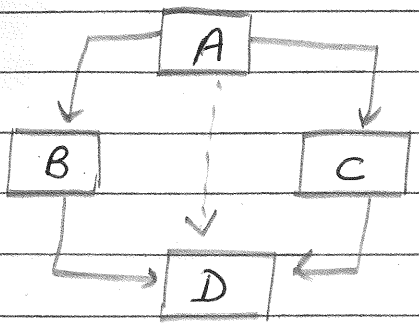
class C : protected test

(i) cannot inherit protected func^n or data.

(ii) protected of test will be inherited in protected of C

(iii) public of test will be inherited in protected of C.

Virtual base classes.



D has two copy of A because of B and C.
compiler cannot understand which copy should run so we use virtual keyword to solve this.

class B : Public virtual A
class C : virtual public A
} means same.

Operator overloading
we can make operator work for user defined classes.
for example, we can overload an operator '+' in a class like string so that we can concatenate two strings by using '+'.

Operators that cannot be used for operator overloading
(a) class member Access operator (., .*)
(b) scope resolution operator (::)
(c) size of
(d) Conditional operator (:?)

→ new operator / delete operator
↓
used to dynamically allocate memory at runtime according to use ie memory is allocated at run time. (used only with pointer)

pointer = new type;

eg: int *p;
p = new int;
p = new int[5]; //array

eg: int *p = new int;
cout<<*p; output
*p = 100; input.

```
int *p = new int[5];

for(i=0; i<5; i++)
 {
    p[i] = 0;
     or                    } same.
   *(p+i) = 0;
```

↓ pointer offset notation.

delete.

syntax: delete p;
delete []p;

If both classes have same func^n disp(). Now base pointer cannot access derived class func^n. It will confuse itself which func^n it should use. It will use base class func^n. Use virtual name in front of base class func^n, now, derived func^n will be used instead.

→ Abstract class and Pure virtual func^n.

↳ one which has pure virtual func^n

↳ virtual func^n which does not have any body

eg:
```
class A
 { virtual void disp() = 0;   // pure virtual func^n
 }
```
class A is called Abstract class.

* Pure virtual func^n and virtual func^n is made in base class only.

Dt.: Pg.: Delta

if someone derives this abstract class, it will be duty of derived class to give definition of pure virtual func^n.
But if it doesnot, derived class also becomes abstract class.

* u cannot make object of abstract class.
ie. A obj; ✗

* u can make pointers of abstract class.
ie A * obj; ✓

```
# include <iostream.h>

class Base
{ public:                                   // abstract class
    virtual void disp() = 0;
};

class D : public Base
{                                           // derived class
  void disp()                               gives def^n of
  {                                           abstract class func^n
    cout << "derived class";
  }
};

int main()
{                          or
  D obj;                                    // object now can be
  obj.disp();                               created.
}
                    int main
                    { B * ptr;    D obj;    // pointers can be
                      ptr = &obj;             made.
                      ptr -> disp();
                    }
```

→ File handling and streams.

< iostream.h >

↳ istream → cin

↳ ostream → cout

use for file handling

< fstream.h >

↳ ifstream : file read

↳ ofstream : file write

↳ fstream : read / write file.

File can be opened in two ways

(i) using constructor func^n of class

(ii) using member func^n open() of class.

(i) opening file using constructor

```
#include < iostream.h >
#include < fstream.h >      ← for opening & closing file.

int main
{  char name [20];
   float cost;                          // writing file.
   ofstream outf ("data");
   cin >> name;
   cin >> cost;

   outf << name;
   outf << cost;
   outf.close ();

   ifstream inf ("data");               // reading file.
   inf >> name;
   inf >> cost;
```

```
    cout << name;
    cout << cost;
    inf.close();
}
```

→ opening file using open() func^n.

```
#include <iostream.h>
#include <fstream.h>

int main()
{ char name [20];
  int cost;

  ofstream fout;                    // object created for writing
  fout.open ("data");                                  data.
  fout << keyboard ;
  fout << 350;
  fout.close();

  ifstream fin;                     // object created for reading
  fin.open ("data");                                   data
  fin >> name;
  fin >> cost;

  cout << name;
  cout << cost;
  fin.close();

  getch();
  return 0;
}
```

open() func^n file mode.

open ("file name", file mode);

ios::in → for reading file

ios::out → for writing in file (open file if exist or create new file)

ios::app → file end (usually to add content at end of line)

ios::ate → file end (can add content at end of line and can modify previous content)

ios::nocreate → if there is no file, no new file will be created, func^n will fail

ios::noreplace → if there is already file, it will not replace existing file.

ios::binary → if you want to create binary file (default: text file)

ios::trunc → if any file already exist, contents are erased and we will be at first line of file.

eg: ofstream outf;

if you want to use two mode ↓

outf.open ("ranan", ios::ate | ios::noreplace);

→ Get() and Put() in C++

↳ reads one character at time.

Get() → ifstream

Put() → ofstream.